

Software Engineering Lab Experiment - 6

Automated UI Testing of [Terminal GIF Maker](#) Using Selenium WebDriver + PyTest

Aim / Objective

The objective of this lab experiment is to perform **automated end-to-end UI testing** of a web application using **Selenium WebDriver** with **PyTest** as the test runner.

The application under test is **Terminal GIF Maker**, a browser-based tool that simulates terminal typing output and exports it as GIF/MP4.

This experiment verifies that:

- The web application loads correctly,
 - UI components behave as expected,
 - Modals (Settings / Export) are displayed **only when intended**, and
 - Key user flows can be automated and validated using assertions.
-

Software / Tools Used

System / Environment

- OS: Windows (PowerShell)
- Python: 3.12.x
- Browser: Google Chrome

Testing Tools

- **Selenium WebDriver** (Python bindings)
 - PyTest (test execution + assertions)
 - pytest-html (optional plugin to generate an HTML report)
 - Python built-in HTTP server (to host the app locally)
-

Theory / Background

What is Selenium?

Selenium is an open-source automation tool used for testing web applications. It controls a real browser (Chrome, Firefox, Edge, etc.) and simulates real user actions such as:

- opening pages,
- clicking buttons,
- typing into inputs,
- checking element visibility,
- validating expected outputs.

Selenium is mainly used for **end-to-end UI testing**, meaning it tests the application exactly as a user would interact with it.

What is Selenium WebDriver?

WebDriver is the Selenium component that provides APIs to control the browser programmatically.

Example WebDriver actions:

- `driver.get(url)` → open a web page
- `find_element(...)` → locate a UI element
- `click()` → click a button
- `is_displayed()` → check visibility
- `WebDriverWait(...)` → wait for dynamic elements to appear/disappear

Why use pytest with Selenium?

pytest provides:

- structured test organization (multiple test cases),
- clear assertions (`assert`),
- automatic test discovery,
- reporting options (HTML/XML),
- easy scalability for larger test suites.

Application Under Test (Step-by-Step Process)

Project: Terminal GIF Maker

Type: Front-end web application

Core Features Tested:

- Page load and title check
- Settings Modal open/close behaviour
- Export Modal open/close behaviour
- Ensuring Export modal does not appear unexpectedly (ex: on “Simulate”)
- Presence of Export GIF and Export MP4 options

Test Strategy and Test Cases

Test Approach

This experiment uses **black-box functional UI testing**, meaning:

- Tests are written based on visible UI behaviour and expected user interactions,
- The internal JavaScript logic is not directly tested,
- The browser UI is treated as the main interface.

Test Cases Implemented

Test ID	Test Case	Expected Result
TC-01	Open homepage	Page loads and title matches “Terminal Gif Maker”
TC-02	Modals hidden on load	Settings modal and Export modal are not visible initially
TC-03	Settings modal open/close	Settings modal becomes visible after click and hides on close
TC-04	Export modal opens only after Export click	Export modal is hidden initially and appears after clicking Export
TC-05	Simulate does not trigger Export modal	Export modal remains hidden after clicking Simulate
TC-06	Export modal options + close	Export modal shows GIF/MP4 buttons and closes properly

Procedure / Step-by-Step Experiment

Step 1: Start a local server to host the project

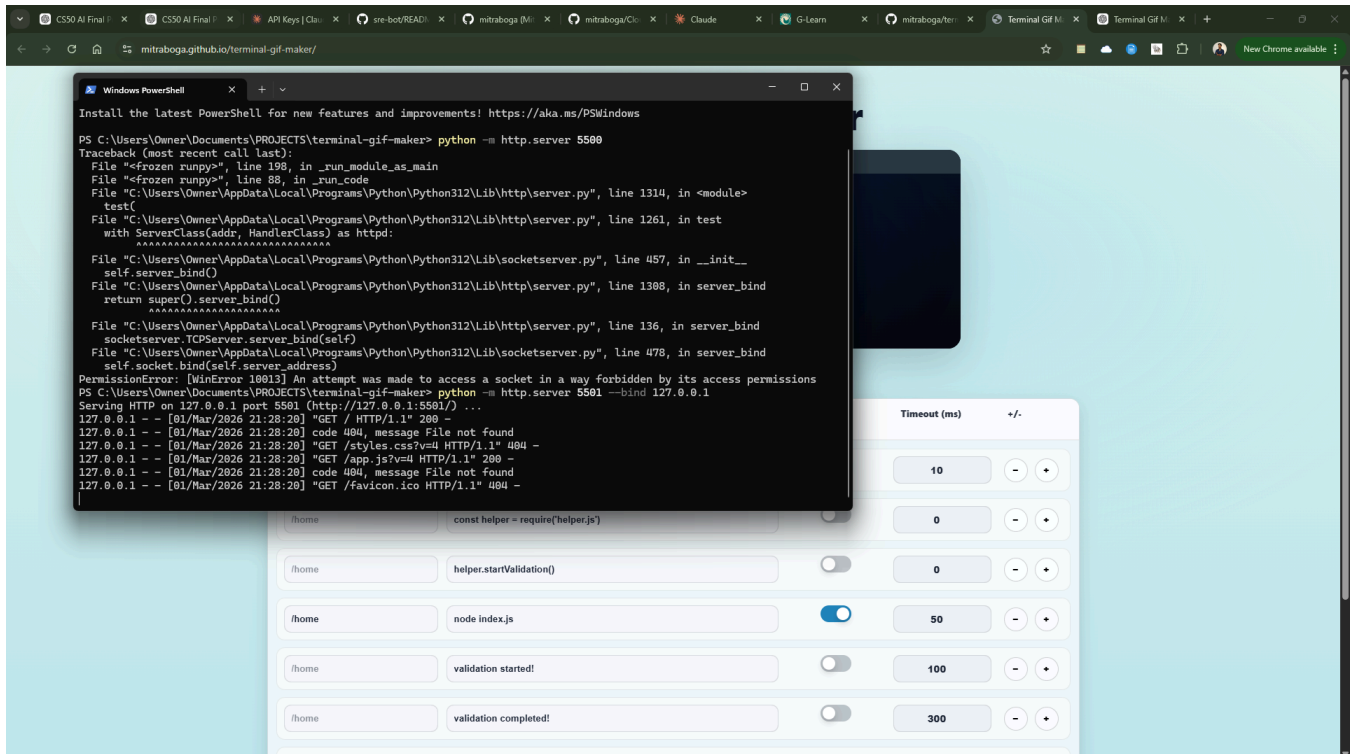
To ensure the web app runs under an HTTP URL (required for stable browser automation and proper browser security behaviour), a Python HTTP server was started.

Command:

```
python -m http.server 5501 --bind 127.0.0.1
```

URL used:

- <http://127.0.0.1:5501>



Terminal Gif Maker

←

↺

127.0.0.1:5501

Terminal Gif Maker

Simulate

Export

Settings

Path

Command

Typing

Animation

Timeout (ms)

+/-

/home	cat index.js	☒
10		
- +		
/home	const helper = require("helpe	☐
0		
- +		
/home	helper.startValidation()	☐
0		
- +		
/home	node index.js	☒
50		
- +		
/home	validation started!	☐
100		
- +		
/home	validation completed!	☐
300		
- +		
/home	command or output text...	☐
50		
- +		
/home	git commit --amend	☒
150		
- +		

☐ Multiline

Tip: Leave **Path** blank to treat a row as output. ANSI works (ex: \u001b[32m).

cat index.js

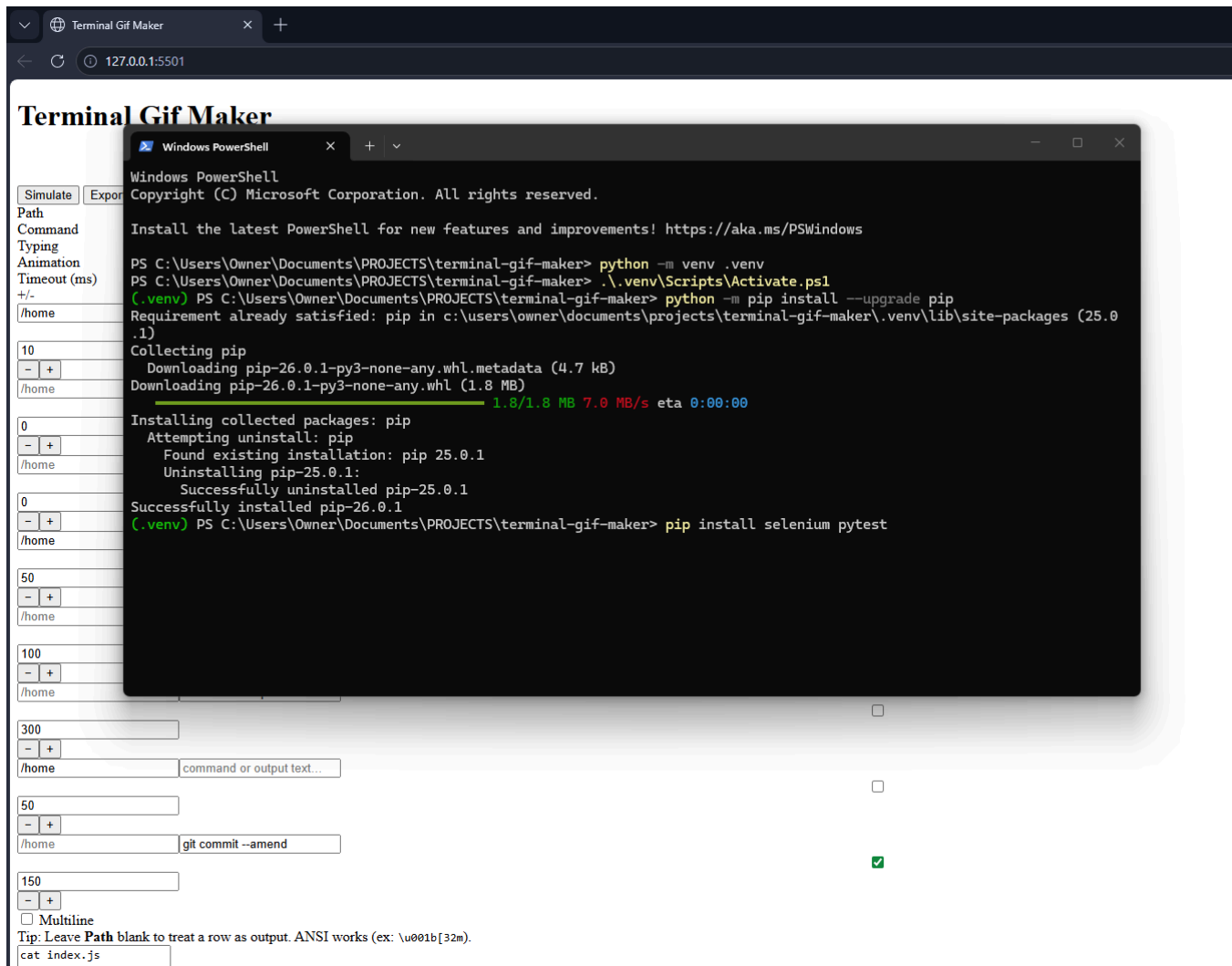
Step 2: Create and activate a virtual environment

A virtual environment isolates dependencies for the project.

Commands:

```
python -m venv .venv
```

..\env\Scripts\Activate.ps1



Step 3: Install required Python packages

Installed Selenium and PyTest:

```
python -m pip install --upgrade pip
```

```
pip install selenium pytest
```

Installed HTML reporting plugin:

```
python -m pip install -U pytest-html
```

```
Windows PowerShell X + v

Found existing installation: pip 25.0.1
Uninstalling pip-25.0.1:
  Successfully uninstalled pip-25.0.1
Successfully installed pip-26.0.1
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pip install selenium pytest
Collecting selenium
  Downloading selenium-4.41.0-py3-none-any.whl.metadata (7.5 kB)
Collecting pytest
  Using cached pytest-9.0.2-py3-none-any.whl.metadata (7.6 kB)
Collecting certifi>=2026.1.4 (from selenium)
  Downloading certifi-2026.2.25-py3-none-any.whl.metadata (2.5 kB)
Collecting trio<1.0,>=0.31.0 (from selenium)
  Downloading trio-0.33.0-py3-none-any.whl.metadata (8.5 kB)
Collecting trio-websocket<1.0,>=0.12.2 (from selenium)
  Downloading trio_websocket-0.12.2-py3-none-any.whl.metadata (5.1 kB)
Collecting typing_extensions<5.0,>=4.15.0 (from selenium)
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting urllib3<3.0,>=2.6.3 (from urllib3[socks]<3.0,>=2.6.3->selenium)
  Using cached urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting websocket-client<2.0,>=1.8.0 (from selenium)
  Downloading websocket_client-1.9.0-py3-none-any.whl.metadata (8.3 kB)
Collecting attrs>=23.2.0 (from trio<1.0,>=0.31.0->selenium)
  Using cached attrs-25.4.0-py3-none-any.whl.metadata (10 kB)
Collecting sortedcontainers (from trio<1.0,>=0.31.0->selenium)
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Collecting idna (from trio<1.0,>=0.31.0->selenium)
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting outcome (from trio<1.0,>=0.31.0->selenium)
  Downloading outcome-1.3.0.post0-py2.py3-none-any.whl.metadata (2.6 kB)
Collecting sniffio>=1.3.0 (from trio<1.0,>=0.31.0->selenium)
  Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting cffi>=1.14 (from trio<1.0,>=0.31.0->selenium)
  Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl.metadata (2.6 kB)
Collecting wsproto>=0.14 (from trio-websocket<1.0,>=0.12.2->selenium)
  Downloading wsproto-1.3.2-py3-none-any.whl.metadata (5.2 kB)
Collecting pysocks!=1.5.7,<2.0,>=1.5.6 (from urllib3[socks]<3.0,>=2.6.3->selenium)
  Downloading PySocks-1.7.1-py3-none-any.whl.metadata (13 kB)
Collecting colorama>=0.4 (from pytest)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Using cached iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging>=22 (from pytest)
  Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Using cached pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Using cached pygments-2.19.2-py3-none-any.whl.metadata (2.5 kB)
Collecting pycparser (from cffi>=1.14->trio<1.0,>=0.31.0->selenium)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Collecting h11<1,>=0.16.0 (from wsproto>=0.14->trio-websocket<1.0,>=0.12.2->selenium)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Downloading selenium-4.41.0-py3-none-any.whl (9.5 MB)
  9.5/9.5 MB 6.0 MB/s 0:00:01
Downloading trio-0.33.0-py3-none-any.whl (510 kB)
```

```
Windows PowerShell

Downloading outcome-1.3.0.post0-py2.py3-none-any.whl.metadata (2.6 kB)
Collecting sniffio-1.3.0 (from trio<1.0,=>0.21.0->selenium)
Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting cffi-1.14 (from trio<1.0,=>0.31.0->selenium)
Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl.metadata (2.6 kB)
Collecting wsproto-0.10 (from trio-websocket<1.0,=>0.12.2->selenium)
Downloading wsproto-1.3.2-py3-none-any.whl.metadata (5.2 kB)
Collecting pysocks-1.5.7, <2.0,=>1.5.6 (from urllib3[socks]<3.0,=>2.6.3->selenium)
Downloading PySocks-1.7.1-py3-none-any.whl.metadata (13 kB)
Collecting colorama-0.4 (from pytest)
Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting iniconfig-1.0.1 (from pytest)
Using cached iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging-22 (from pytest)
Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pluggy-2, >=1.5 (from pytest)
Using cached pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments-2.7.2 (from pytest)
Using cached pygments-2.19.2-py3-none-any.whl.metadata (2.5 kB)
Collecting pycparser (from cffi-1.14->trio<1.0,=>0.31.0->selenium)
Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Collecting h11<1, >=0.16.0 (from wsproto-0.10->trio-websocket<1.0,=>0.12.2->selenium)
Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Downloading selenium-4.41.0-py3-none-any.whl (9.5 MB)
8.5/9.5 MB 6.0 MB/s 0:00:01

Downloading trio-0.33.0-py3-none-any.whl (510 kB)
Downloading trio_websocket-0.12.2-py3-none-any.whl (21 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Using cached urllib3-2.6.3-py3-none-any.whl (131 kB)
Downloading PySocks-1.7.1-py3-none-any.whl (16 kB)
Downloading websocket_client-1.9.0-py3-none-any.whl (82 kB)
Using cached pytest-9.0.2-py3-none-any.whl (374 kB)
Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Using cached attrs-25.4.0-py3-none-any.whl (67 kB)
Downloading certifi-2026.2.25-py3-none-any.whl (153 kB)
Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl (183 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading outcome-1.3.0.post0-py2.py3-none-any.whl (10 kB)
Using cached packaging-26.0-py3-none-any.whl (1.2 MB)
Using cached pygments-2.19.2-py3-none-any.whl (1.2 MB)
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Downloading wsproto-1.3.2-py3-none-any.whl (24 kB)
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Downloading pycparser-3.0-py3-none-any.whl (40 kB)
Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Installing collected packages: sortedcontainers, websocket-client, urllib3, typing_extensions, sniffio, pysocks, pygments, pycparser, pluggy, packaging, iniconfig, idna, h11, colorama, certifi, attrs, wsproto,
pytest, outcome, cffi, trio, trio-websocket, selenium
Successfully installed attrs-25.4.0 certifi-2026.2.25 cffi-2.0.0 colorama-0.4.6 h11-0.16.0 idna-3.11 iniconfig-2.3.0 outcome-1.3.0.post0 packaging-26.0 pluggy-1.6.0 pycparser-3.0 pygments-2.19.2 pysocks-1.7.1
pytest-9.0.2 selenium-4.41.0 sniffio-1.3.1 sortedcontainers-2.4.0 trio-0.33.0 trio-websocket-0.12.2 typing_extensions-4.15.0 urllib3-2.6.3 websocket-client-1.9.0 wsproto-1.3.2
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> python -c "import selenium; print(selenium.__version__)"
4.41.0
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> |
```

```
Windows PowerShell

Downloading trio-0.33.0-py3-none-any.whl (510 kB)
Downloading trio_websocket-0.12.2-py3-none-any.whl (21 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Using cached urllib3-2.6.3-py3-none-any.whl (131 kB)
Downloading PySocks-1.7.1-py3-none-any.whl (16 kB)
Downloading websocket_client-1.9.0-py3-none-any.whl (82 kB)
Using cached pytest-9.0.2-py3-none-any.whl (374 kB)
Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Using cached attrs-25.4.0-py3-none-any.whl (67 kB)
Downloading certifi-2026.2.25-py3-none-any.whl (153 kB)
Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl (183 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading outcome-1.3.0.post0-py2.py3-none-any.whl (10 kB)
Using cached packaging-26.0-py3-none-any.whl (1.2 MB)
Using cached pygments-2.19.2-py3-none-any.whl (1.2 MB)
Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Downloading wsproto-1.3.2-py3-none-any.whl (24 kB)
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Downloading pycparser-3.0-py3-none-any.whl (40 kB)
Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Installing collected packages: sortedcontainers, websocket-client, urllib3, typing_extensions, sniffio, pysocks, pygments, pycparser, pluggy, packaging, iniconfig, idna, h11, colorama, certifi, attrs, wsproto,
pytest, outcome, cffi, trio, trio-websocket, selenium
Successfully installed attrs-25.4.0 certifi-2026.2.25 cffi-2.0.0 colorama-0.4.6 h11-0.16.0 idna-3.11 iniconfig-2.3.0 outcome-1.3.0.post0 packaging-26.0 pluggy-1.6.0 pycparser-3.0 pygments-2.19.2 pysocks-1.7.1
pytest-9.0.2 selenium-4.41.0 sniffio-1.3.1 sortedcontainers-2.4.0 trio-0.33.0 trio-websocket-0.12.2 typing_extensions-4.15.0 urllib3-2.6.3 websocket-client-1.9.0 wsproto-1.3.2
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> python -c "import selenium; print(selenium.__version__)"
4.41.0
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q
no tests ran in 0.02s
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q
1 passed in 7.65s
[100%]
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> |
```

```
File Edit Selection View Go Run Terminal Help
EXPLORER
TERMINAL-GIF-MAKER
> .pytest_cache
> .venv
> tests
> __pycache__
test_smoke.py
JS app.js
JS gif.worker.js
index.html
# style.css

tests > test_smoke.py > test_open_homepage
1 from selenium import webdriver
2
3 def test_open_homepage():
4     driver = webdriver.Chrome()
5     try:
6         driver.get("http://127.0.0.1:5501")
7         assert "Terminal" in driver.title or driver.title != ""
8     finally:
9         driver.quit()
```


Step 4: Write Selenium + PyTest test scripts

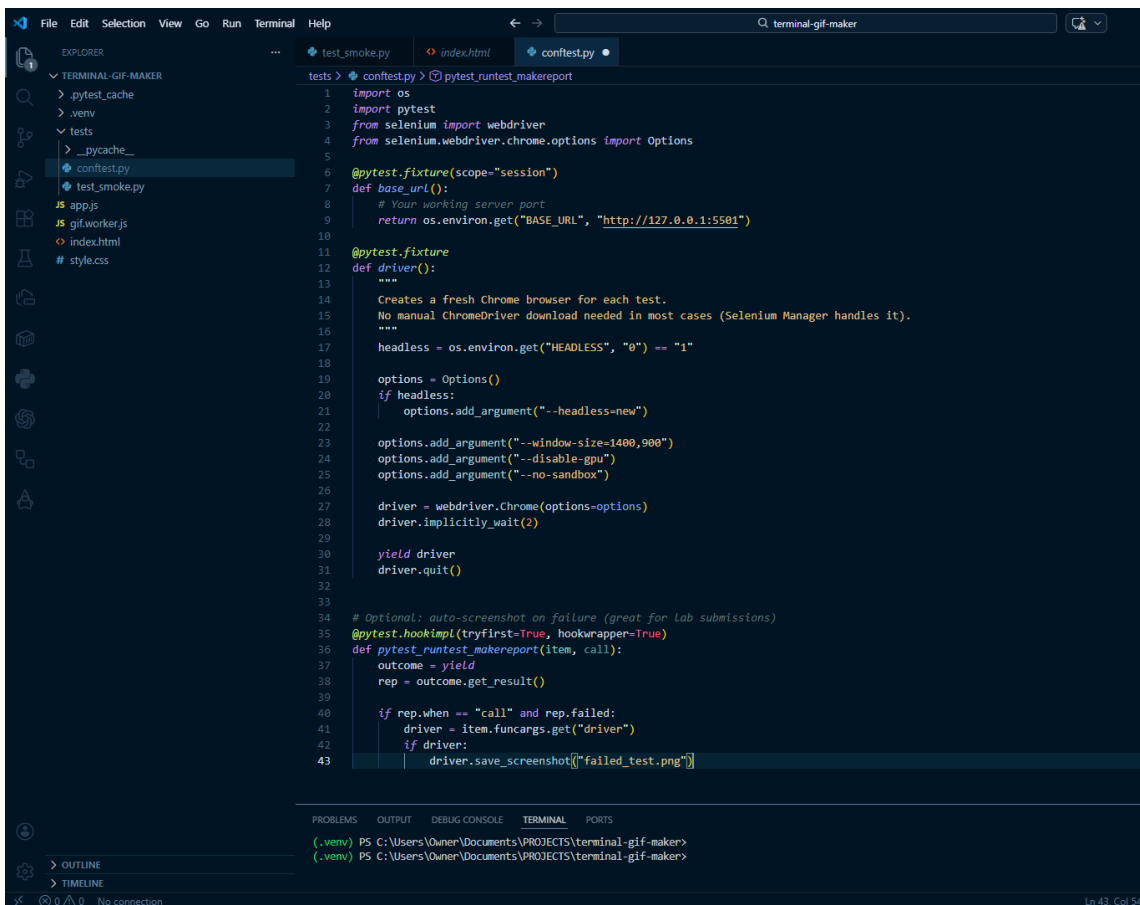
A `tests/` directory was created, and test scripts were written to automate UI behaviour.

The tests locate elements using stable IDs from the application HTML, such as:

- `btnSettings, settingsModal, closeSettings`
- `btnExport, exportModal, closeExport`
- `btnExportGif, btnExportMp4`
- `btnSim`

Key validations were performed using assertions like:

- The modal should be hidden on load
- The modal should appear only when the correct button is clicked
- Required export options must be visible



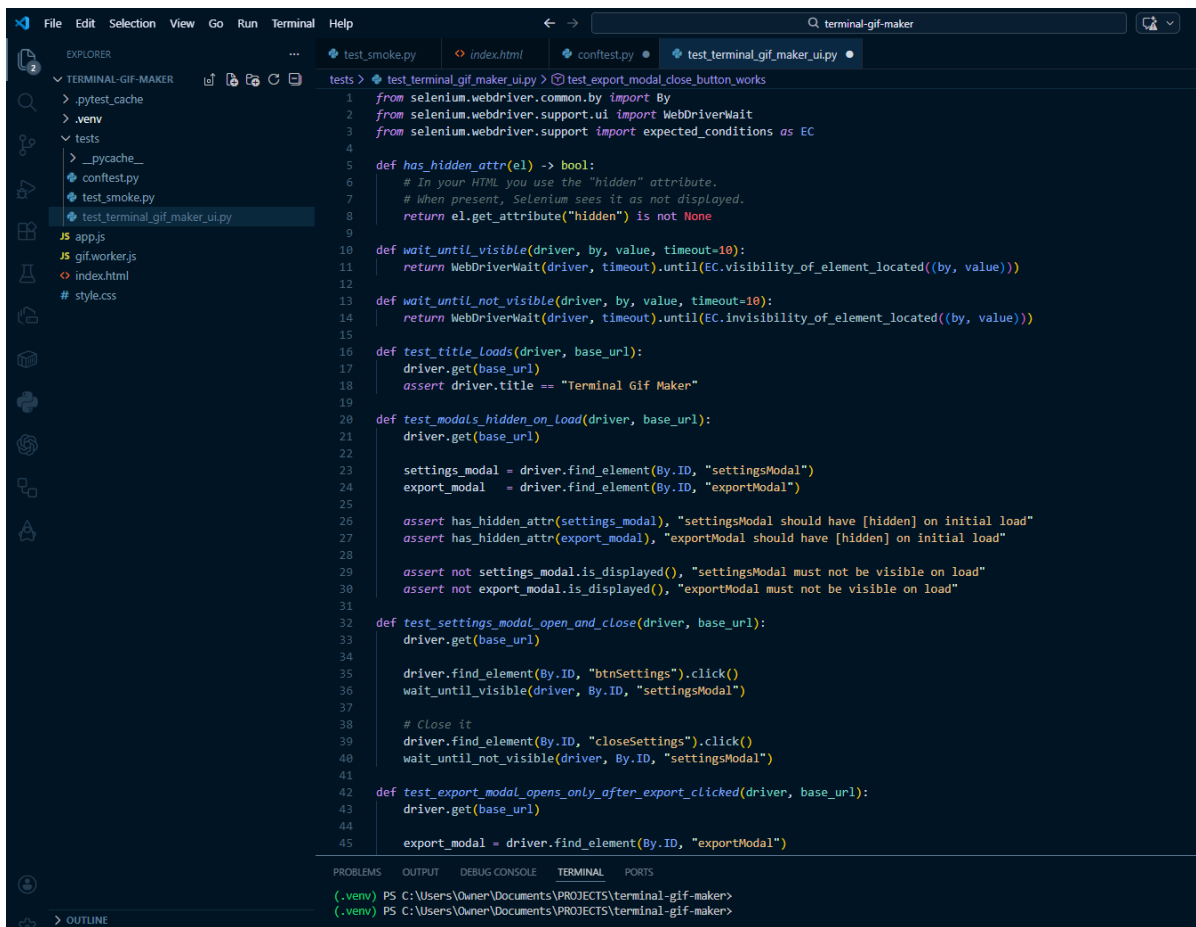
The screenshot shows a VS Code editor with a project named 'terminal-gif-maker'. The Explorer sidebar on the left shows the file structure, including a 'tests' directory. The main editor window displays the file 'conftest.py' with the following Python code:

```
1 import os
2 import pytest
3 from selenium import webdriver
4 from selenium.webdriver.chrome.options import Options
5
6 @pytest.fixture(scope="session")
7 def base_url():
8     # Your working server port
9     return os.environ.get("BASE_URL", "http://127.0.0.1:5501")
10
11 @pytest.fixture
12 def driver():
13     """
14     Creates a fresh Chrome browser for each test.
15     No manual ChromeDriver download needed in most cases (Selenium Manager handles it).
16     """
17     headless = os.environ.get("HEADLESS", "0") == "1"
18
19     options = Options()
20     if headless:
21         options.add_argument("--headless=new")
22
23     options.add_argument("--window-size=1400,900")
24     options.add_argument("--disable-gpu")
25     options.add_argument("--no-sandbox")
26
27     driver = webdriver.Chrome(options=options)
28     driver.implicitly_wait(2)
29
30     yield driver
31     driver.quit()
32
33 # Optional: auto-screenshot on failure (great for lab submissions)
34 @pytest.hookimpl(tryfirst=True, hookwrapper=True)
35 def pytest_runtest_makereport(item, call):
36     outcome = yield
37     rep = outcome.get_result()
38
39     if rep.when == "call" and rep.failed:
40         driver = item.funcargs.get("driver")
41         if driver:
42             driver.save_screenshot("failed_test.png")
```

The bottom of the editor shows the 'TERMINAL' panel with the following commands:

```
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker>
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker>
```

The status bar at the bottom indicates 'Ln 43, Col 54'.



```
Installing collected packages: sortedcontainers, websocket-client, urllib3, typing_extensions, sniffio, pysocks, pygments, pycparser, pluggy, packaging, iniconfig, idna, h11, colorama, certifi, attrs, wsproto,
pytest, outcome, cffi, trio, trio-websocket, selenium
Successfully installed attrs-25.4.0 certifi-2026.2.25 cffi-2.0.0 colorama-0.4.6 h11-0.16.0 idna-3.11 iniconfig-2.3.0 outcome-1.3.0 post0 packaging-26.0 pluggy-1.6.0 pycparser-3.0 pygments-2.19.2 pysocks-1.7.1
pytest-9.0.2 selenium-4.41.0 sniffio-1.3.1 sortedcontainers-2.4.0 trio-0.33.0 trio-websocket-0.12.2 typing_extensions-4.15.0 urllib3-2.6.3 websocket-client-1.9.0 wsproto-1.3.2
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> python -c "import selenium; print(selenium.__version__)"
4.41.0
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q
no tests ran in 0.82s
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q
1 passed in 7.65s [100%]
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> $env:BASE_URL="http://127.0.0.1:5581"
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q
1 passed in 3.53s [100%]
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> $env:HEADLESS="1"
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q
1 passed in 3.77s [100%]

(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> python -m pip install -U pytest-html
Collecting pytest-html
  Downloading pytest_html-4.2.0-py3-none-any.whl.metadata (4.0 kB)
Collecting Jinja2>=3 (from pytest-html)
  Using cached Jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting pytest-metadata>=2 (from pytest-html)
  Downloading pytest_metadata-3.1.1-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: pytest>=7 in .\venv\Lib\site-packages (from pytest-html) (9.0.2)
Collecting MarkupSafe>=2.0 (from Jinja2>=3->pytest-html)
  Downloading MarkupSafe-3.0.3-cp312-cp312-win_amd64.whl.metadata (2.8 kB)
Requirement already satisfied: colorama>=0.4 in .\venv\Lib\site-packages (from pytest>=7->pytest-html) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in .\venv\Lib\site-packages (from pytest>=7->pytest-html) (2.3.0)
Requirement already satisfied: packaging>=22 in .\venv\Lib\site-packages (from pytest>=7->pytest-html) (26.0)
Requirement already satisfied: pluggy>=2, >=1.5 in .\venv\Lib\site-packages (from pytest>=7->pytest-html) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in .\venv\Lib\site-packages (from pytest>=7->pytest-html) (2.19.2)
Downloading pytest_html-4.2.0-py3-none-any.whl (23 kB)
Using cached Jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading MarkupSafe-3.0.3-cp312-cp312-win_amd64.whl (115 kB)
Downloading pytest_metadata-3.1.1-py3-none-any.whl (11 kB)
Installing collected packages: MarkupSafe, pytest-metadata, Jinja2, pytest-html
Successfully installed MarkupSafe-3.0.3 Jinja2-3.1.6 pytest-html-4.2.0 pytest-metadata-3.1.1
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest --help | Select-String html

--html=path          create html report file at given path.
--self-contained-html create a self-contained html file containing all
                     the html report.
                     the HTML report will be generated after each test

(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q --html=selenium_report.html --self-contained-html
----- Generated html report: file:///C:/Users/Owner/Documents/PROJECTS/terminal-gif-maker/selenium_report.html [100%]
1 passed in 3.63s
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> pytest -q --junitxml=selenium_report.xml
1 passed in 3.43s [100%]
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> |
```

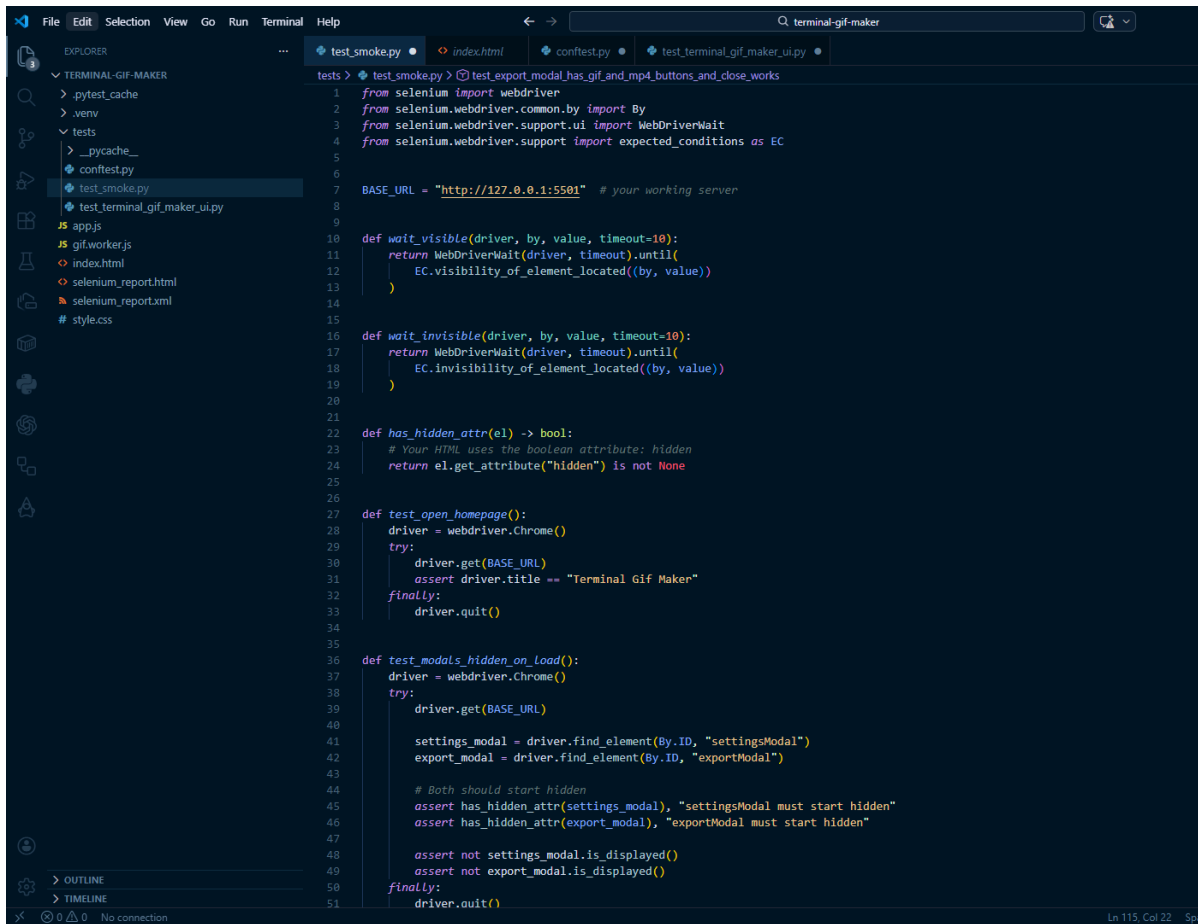
Step 5: Execute tests using PyTest

Tests were executed using:

```
pytest -q -vv
```

Result:

- tests executed successfully and passed.



```
File Edit Selection View Go Run Terminal Help
test_smoke.py index.html conftest.py test_terminal_gif_maker_ui.py
TERMINAL-GIF-MAKER
> pytest_cache
> .venv
> tests
  > _pycache_
    test_smoke.py
    test_terminal_gif_maker_ui.py
  app.js
  gif.worker.js
  index.html
  selenium_report.html
  selenium_report.xml
  # style.css

1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5
6
7 BASE_URL = "http://127.0.0.1:5501" # your working server
8
9
10 def wait_visible(driver, by, value, timeout=10):
11     return WebDriverWait(driver, timeout).until(
12         EC.visibility_of_element_located((by, value))
13     )
14
15
16 def wait_invisible(driver, by, value, timeout=10):
17     return WebDriverWait(driver, timeout).until(
18         EC.invisibility_of_element_located((by, value))
19     )
20
21
22 def has_hidden_attr(el) -> bool:
23     # Your HTML uses the boolean attribute: hidden
24     return el.get_attribute("hidden") is not None
25
26
27 def test_open_homepage():
28     driver = webdriver.Chrome()
29     try:
30         driver.get(BASE_URL)
31         assert driver.title == "Terminal Gif Maker"
32     finally:
33         driver.quit()
34
35
36 def test_modals_hidden_on_load():
37     driver = webdriver.Chrome()
38     try:
39         driver.get(BASE_URL)
40
41         settings_modal = driver.find_element(By.ID, "settingsModal")
42         export_modal = driver.find_element(By.ID, "exportModal")
43
44         # Both should start hidden
45         assert has_hidden_attr(settings_modal), "settingsModal must start hidden"
46         assert has_hidden_attr(export_modal), "exportModal must start hidden"
47
48         assert not settings_modal.is_displayed()
49         assert not export_modal.is_displayed()
50     finally:
51         driver.quit()
```

```
Platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker\.venv\Scripts\python.exe
cachedir: pytest_cache
metadata: {'Python': '3.12.10', 'Platform': 'Windows-11-10.0.26280-SP0', 'Packages': {'pytest': '9.0.2', 'pluggy': '1.6.0'}, 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}, 'JAVA_HOME': 'C:\\Program Files\\Eclipse Adoptium\\jdk-25.0.2.10-hotspot\\'}
rootdir: C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker
plugins: html-4.2.0, metadata-3.1.1
collected 6 items

tests/test_smoke.py::test_open_homepage PASSED [ 16%]
tests/test_smoke.py::test_modals_hidden_on_load PASSED [ 33%]
tests/test_smoke.py::test_settings_modal_open_and_close FAILED [ 50%]
tests/test_smoke.py::test_export_modal_opens_only_after_export_clicked FAILED [ 66%]
tests/test_smoke.py::test_simulate_does_not_open_export_modal PASSED [ 83%]
tests/test_smoke.py::test_export_modal_has_gif_and_mp4_buttons_and_close_works FAILED [100%]

===== FAILURES =====
test_settings_modal_open_and_close

def test_settings_modal_open_and_close():
    driver = webdriver.Chrome()
    try:
        driver.get(BASE_URL)

        driver.find_element(By.ID, "btnSettings").click()
        wait_visible(driver, By.ID, "settingsModal")
    >
        driver.find_element(By.ID, "closeSettings").click()

tests/test_smoke.py:62:
-----
.venv\Lib\site-packages\selenium\webdriver\remote\webelement.py:110: in click
    self.execute(Command.CLICK_ELEMENT)
.venv\Lib\site-packages\selenium\webdriver\remote\webelement.py:508: in _execute
    return self._parent.execute(command, params)
.venv\Lib\site-packages\selenium\webdriver\remote\webdriver.py:446: in execute
    self.error_handler.check_response(response)
```



```

E      0x7fff7a2c91e47
E      0x7fff8af78e8d7
E      0x7fff8b162c48c

.venv\Lib\site-packages\selenium\webdriver\remote\errorhandler.py:232: ElementNotInteractableException
----- test_export_modal_opens_only_after_export_clicked -----
def test_export_modal_opens_only_after_export_clicked():
    driver = webdriver.Chrome()
    try:
        driver.get(BASE_URL)

        export_modal = driver.find_element(By.ID, "exportModal")
        assert has_hidden_attr(export_modal)
        assert not export_modal.is_displayed()

        driver.find_element(By.ID, "btnExport").click()
        wait_visible(driver, By.ID, "exportModal")
>
tests\test_smoke.py:78:
-----
tests\test_smoke.py:11: in wait_visible
    return WebDriverWait(driver, timeout).until(
-----
self = <selenium.webdriver.support.wait.WebDriverWait (session="elec85ffe0c532e702f99faf0bf384fb")>, method = <function visibility_of_element_located.<locals>._predicate at 0x00000144EA76B6A8>, message = ''

def until(self, method: Callable[[D], Literal[False] | T], message: str = "") -> T:
    """Wait until the method returns a value that is not False.

    Calls the method provided with the driver as an argument until the
    return value does not evaluate to ``False``.

    Args:
        method: A callable object that takes a WebDriver instance as an
            argument.
        message: Optional message for TimeoutException.

    Returns:
        The result of the last call to 'method'.

    Raises:
        TimeoutException: If 'method' does not return a truthy value within
            the WebDriverWait object's timeout.

    Example:
    >>> from selenium.webdriver.common.by import By
    >>> from selenium.webdriver.support.ui import WebDriverWait
    >>> from selenium.webdriver.support import expected_conditions as EC
    >>>
    >>> # Wait until an element is visible on the page
    >>> wait = WebDriverWait(driver, 10)
    >>> element = wait.until(EC.visibility_of_element_located((By.ID, "exampleId")))
    >>> print(element.text)

```

```

screen = None
stacktrace = None

end_time = time.monotonic() + self._timeout
while True:
    try:
        value = method(self._driver)
        if value:
            return value
        except self._ignored_exceptions as exc:
            screen = getattr(exc, "screen", None)
            stacktrace = getattr(exc, "stacktrace", None)
            if time.monotonic() > end_time:
                break
            time.sleep(self._poll)
    raise TimeoutException(message, screen, stacktrace)
E selenium.common.exceptions.TimeoutException: Message:

.venv\Lib\site-packages\selenium\webdriver\support\wait.py:121: TimeoutException
----- test_export_modal_has_gif_and_mp4_buttons_and_close_works -----
def test_export_modal_has_gif_and_mp4_buttons_and_close_works():
    driver = webdriver.Chrome()
    try:
        driver.get(BASE_URL)

        driver.find_element(By.ID, "btnExport").click()
        wait_visible(driver, By.ID, "exportModal")
>
tests\test_smoke.py:103:
-----
tests\test_smoke.py:11: in wait_visible
    return WebDriverWait(driver, timeout).until(
-----
self = <selenium.webdriver.support.wait.WebDriverWait (session="009b3a7702ec7d036e0cc5443aeedbce")>, method = <function visibility_of_element_located.<locals>._predicate at 0x00000144EA76B6A8>, message = ''

def until(self, method: Callable[[D], Literal[False] | T], message: str = "") -> T:
    """Wait until the method returns a value that is not False.

    Calls the method provided with the driver as an argument until the
    return value does not evaluate to ``False``.

    Args:
        method: A callable object that takes a WebDriver instance as an
            argument.
        message: Optional message for TimeoutException.

    Returns:
        The result of the last call to 'method'.

    Raises:
        TimeoutException: If 'method' does not return a truthy value within
            the WebDriverWait object's timeout.

```

```
def until(self, method: Callable[[D], Literal[False] | T], message: str = "") -> T:
    """Wait until the method returns a value that is not False.

    Calls the method provided with the driver as an argument until the
    return value does not evaluate to ``False``.

    Args:
        method: A callable object that takes a WebDriver instance as an
            argument.
        message: Optional message for TimeoutException.

    Returns:
        The result of the last call to 'method'.

    Raises:
        TimeoutException: If 'method' does not return a truthy value within
            the WebDriverWait object's timeout.

    Example:
        >>> from selenium.webdriver.common.by import By
        >>> from selenium.webdriver.support.ui import WebDriverWait
        >>> from selenium.webdriver.support import expected_conditions as EC
        >>>
        >>> # Wait until an element is visible on the page
        >>> wait = WebDriverWait(driver, 10)
        >>> element = wait.until(EC.visibility_of_element_located((By.ID, "exampleId")))
        >>> print(element.text)
    """
    screen = None
    stacktrace = None

    end_time = time.monotonic() + self._timeout
    while True:
        try:
            value = method(self._driver)
            if value:
                return value
        except self._ignored_exceptions as exc:
            screen = getattr(exc, "screen", None)
            stacktrace = getattr(exc, "stacktrace", None)
            if time.monotonic() > end_time:
                break
        time.sleep(self._poll)
    raise TimeoutException(message, screen, stacktrace)
E selenium.common.exceptions.TimeoutException: Message:

.venv\Lib\site-packages\selenium\webdriver\support\wait.py:121: TimeoutException
===== short test summary info =====
FAILED tests/test_smoke.py::test_settings_modal_open_and_close - selenium.common.exceptions.ElementNotInteractableException: Message: element not interactable
FAILED tests/test_smoke.py::test_export_modal_opens_only_after_export_clicked - selenium.common.exceptions.TimeoutException: Message:
FAILED tests/test_smoke.py::test_export_modal_has_gif_and_mp4_buttons_and_close_works - selenium.common.exceptions.TimeoutException: Message:
===== 3 failed, 3 passed in 43.04s =====
(.venv) PS C:\Users\Owner\Documents\PROJECTS\terminal-gif-maker> |
```

Step 6: Generate test reports

HTML Report:

```
pytest -q --html=selenium_report.html --self-contained-html
```

JUnit XML Report:

```
pytest -q --junitxml=selenium_report.xml
```

selenium_report.html

Report generated on 01-Mar-2025 at 22:08:09 by [pytest-html](#) v4.2.0

Environment

Python	3.12.10
Platform	Windows-11-10.0.26200-SP0
Packages	- pytest: 9.0.2 - pluggy: 1.5.0
Plugins	- html: 4.2.0 - metadata: 3.1.1
JAVA_HOME	C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot

Summary

1 test took 00:00:04.

(Un)check the boxes to filter the results.

☐ Failed ☒ 1 Passed ☐ 0 Skipped ☐ 0 Expected failures ☐ 0 Unexpected passes ☐ 0 Errors ☐ 0 Reruns ☐ 0 Retired

Result	Test	Duration	Links
Passed	tests/test_smoke.py::test_open_homepage	00:00:04	

[Show all details](#) / [Hide all details](#)

7. Observations / Results

- Selenium successfully automated Chrome and executed UI test cases.
 - There were 3 PASSES and 3 FAILS. This Selenium test tool was useful in testing, and now I am aware of the functionalities that failed during testing, and can go back to fix them, as it will enhance and refine my [Terminal GIF Maker Project!](#)
 - Reports generated successfully:
 - `selenium_report.html`
 - `selenium_report.xml`
-

8. Conclusion

This lab experiment successfully demonstrated **end-to-end UI automation testing** using **Selenium WebDriver** and **PyTest** on the Terminal GIF Maker web application.

The automated test suite validated essential UI behaviors including correct modal display logic and user interaction flow, ensuring the application behaves as intended.

The experiment also produced professional test artifacts (HTML and XML reports), which can be used for documentation, grading evidence, and future regression testing.

9. Viva / Short Q&A

Q1. Why did we serve the project using `http.server` instead of opening the HTML directly?

Because browsers restrict certain features under `file://` (workers/blobs/download behaviour). Hosting through `http://127.0.0.1` makes behaviour consistent and testable.

Q2. What kind of testing does Selenium perform?

End-to-end UI (black-box) testing by automating real browser interaction.

Q3. What is the benefit of running tests headless?

Faster execution and suitable for CI/CD pipelines where no UI is available.

Q4. Why use explicit waits (`WebDriverWait`) in Selenium?

To handle dynamic UI elements reliably and avoid flakiness due to timing issues.